

Daily AI Engineering Report

WST AI

AI Engineer

September 22, 2026

Project: WST AI **Role:** AI Engineer **Date:** September 22, 2026 **Focus:** AI API Deployment, Testing, and Backend Integration Preparation

1. Overview

Today, I completed the deployment and online validation of the WST AI machine learning API.

The AI models were integrated with a FastAPI application, containerized using Docker, connected to the GitHub repository, and successfully deployed to Vercel.

The deployed API is now accessible online and can be consumed by the Backend team through REST API endpoints.

2. Work Completed

2.1 FastAPI Integration

Verified and prepared the FastAPI application containing two main AI services:

- **Student Training Risk Prediction**
- **Inventory Reorder Prediction**

The API exposes the following endpoints:

```
GET /  
POST /predict/student-risk  
POST /predict/inventory-reorder
```

The API also provides automatically generated Swagger/OpenAPI documentation.

2.2 Student Risk AI Endpoint

Verified the student risk prediction endpoint using the following input:

```
{  
  "student_id": "1",  
  "attendance_status": "Present",  
  "task_assessment_result": "Pass",  
  "time_on_task_minutes": 30  
}
```

The API successfully returned:

```
{
```

```
    "student_id": "1",
    "ai_training_risk_flag": "LOW_RISK_ON_TRACK"
}
```

The endpoint was successfully tested with an HTTP **200 OK** response.

2.3 Inventory Reorder AI Endpoint

Tested the inventory reorder prediction endpoint using:

```
{
  "part_sku": "BRK-001",
  "inventory_on_hand_qty": 5,
  "inventory_min_reorder_level": 10,
  "part_issued_qty": 8,
  "part_unit_cost": 150.0
}
```

The AI successfully returned:

```
{
  "part_sku": "BRK-001",
  "ai_reorder_status": "REORDER_RECOMMENDED"
}
```

The endpoint was successfully tested online with an HTTP **200 OK** response.

3. Docker Deployment Preparation

Verified the Docker-based deployment configuration.

The Dockerfile was configured to:

- Use Python 3.11.
- Install the required Python dependencies.
- Copy the application files into the container.
- Run the FastAPI application using Uvicorn.
- Support the deployment platform's dynamic PORT environment variable.

The Docker image was successfully built and tested locally.

4. GitHub Integration

The project was synchronized with the GitHub repository:

Repository: Wst-project-pA6/AI-ML-

The deployment-related changes were committed and pushed to the main branch.

The final deployment fix was committed as:

```
253150d
Fix Vercel SQLite write path
```

Vercel automatically detected the new commit and created a new production deployment.

5. SQLite / Vercel Deployment Issue

During the first online test, the Inventory Reorder endpoint returned HTTP **500**.

The AI prediction itself was successful, but the application failed when attempting to write the prediction log to the SQLite database.

The error was:

```
sqlite3.OperationalError:  
attempt to write a readonly database
```

Resolution

The database configuration was updated so that:

- Local development continues to use the local `wst_ai.db`.
- Vercel uses a writable temporary database location under `/tmp`.

After the change, the application was tested locally and all existing tests passed.

6. Automated Testing

The project test suite was executed after the database configuration change.

Result:

4 passed, 1 warning

The four tests covered:

- Root endpoint
- Successful Student Risk prediction
- Successful Inventory Reorder prediction
- Student Risk invalid-data validation

The warning was a dependency deprecation warning and did not cause any test failure.

7. Vercel Deployment

The FastAPI application was successfully deployed to Vercel.

Production Status

```
Status: Ready  
Environment: Production  
Branch: main
```

Production API

<https://wst-ai.vercel.app>

API Documentation

<https://wst-ai.vercel.app/docs>

The production deployment was tested successfully after the SQLite configuration fix.

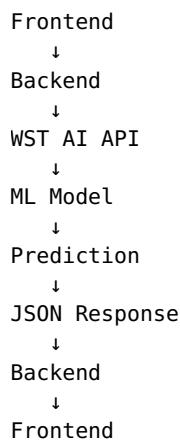
8. Backend Integration Preparation

The API is now ready for integration with the Backend system.

The Backend team does not need to install or directly access the trained ML models.

Instead, the Backend can communicate with the AI service using HTTP POST requests.

The integration flow is:



The Backend team can use the Swagger documentation to understand and test the available endpoints.

9. AI Engineer Responsibilities Completed

Today's work covered several stages of the AI engineering lifecycle:

- ML model integration
 - FastAPI API development
 - API endpoint testing
 - Docker configuration
 - GitHub version control
 - Cloud deployment
 - Production debugging
 - Database deployment compatibility
 - Automated testing
 - Online API validation
 - Backend integration preparation
-

10. Current Status

Component	Status
Student Risk Model	Completed
Inventory Reorder Model	Completed
FastAPI API	Completed
Docker Configuration	Completed
GitHub Integration	Completed

Automated Tests	Passed
Vercel Deployment	Ready
Student Risk API	Online / Tested
Inventory Reorder API	Online / Tested
Swagger Documentation	Available
Backend Integration	Ready

11. Next Steps

The next phase is to integrate the deployed AI API with the main Backend system.

The Backend team will consume:

```
POST /predict/student-risk  
POST /predict/inventory-reorder
```

For a production system, persistent prediction logging should eventually be moved from the temporary Vercel filesystem to a persistent managed database or handled by the main Backend database.

Conclusion

Today, the WST AI models were successfully transformed from locally trained machine learning models into an accessible online AI service.

The FastAPI application is deployed on Vercel, both prediction endpoints have been tested successfully, automated tests are passing, and the API is ready for Backend integration.

Production API: <https://wst-ai.vercel.app>

Swagger Documentation: <https://wst-ai.vercel.app/docs>